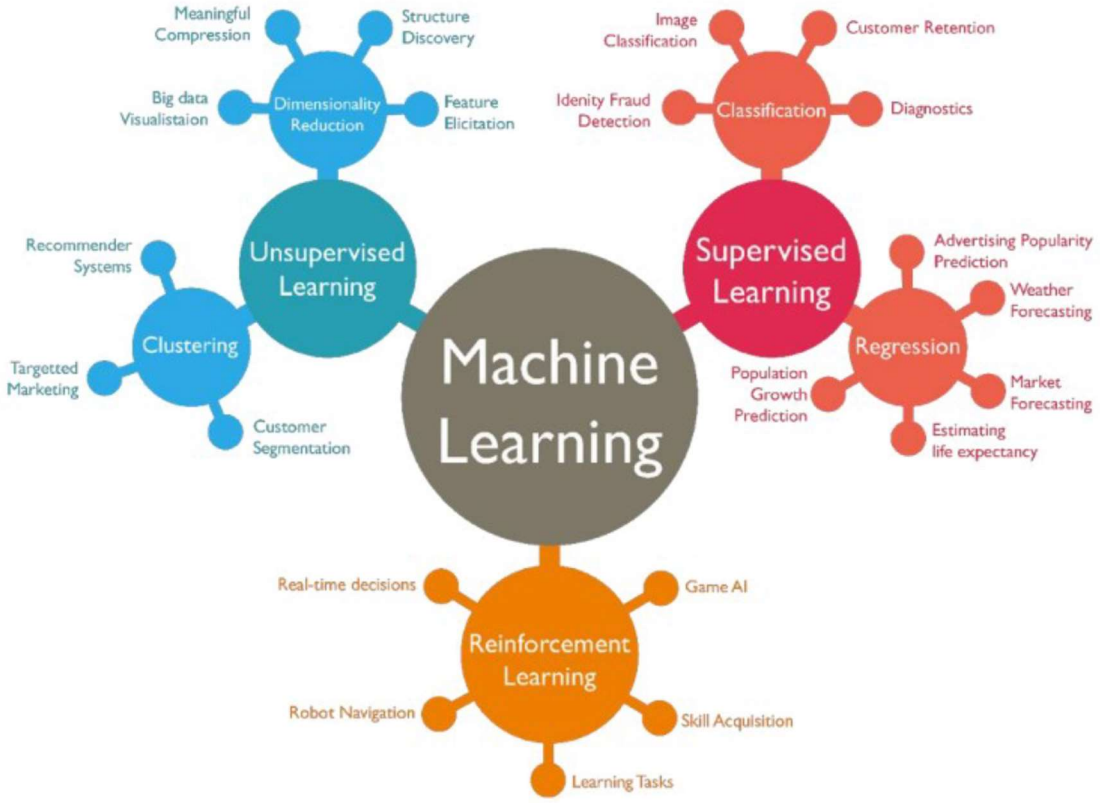
Aprendizaje supervisado

Introducción 2
Aprendizaje supervisado 3
Regresión lineal 5
Regresión logística 9
Support vector machines (SVM) 10
SVC (clasificación) 13
SVR. Regresión 14
Árboles de decisión 14

Introducción

El aprendizaje supervisado utiliza datos etiquetados (que contienen la variable objetivo) para que el modelo prediga resultados, mientras que el aprendizaje no supervisado utiliza datos no etiquetados para que el modelo descubra patrones por sí mismo.

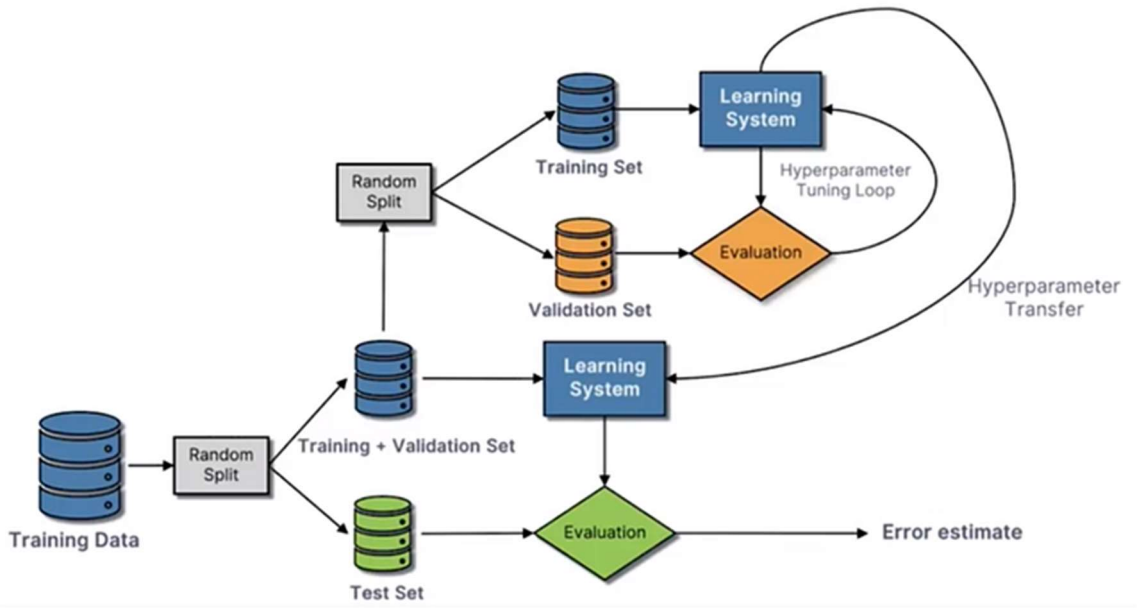


Para decidir qué tipo de algoritmo usar hay que tener en cuenta si los datos están etiquetados o sin etiquetar y tener muy claros los objetivos. Es importante conocer el tipo de problema a la hora de decidir si se quiere crear un modelo de predicción o si se quiere descubrir nueva información valiosa o patrones ocultos en los datos.

Aún así hay muchos algoritmos y elegir el mejor no es una tarea inmediata.

Para ello:

1. Seleccionamos un conjunto de algoritmos
2. Se entrena cada algoritmo con los datos de entrenamiento
3. Se evalúa el rendimiento
4. Se ajustan los hiperparámetros para cada algoritmo
5. Se comparan los modelos utilizando métricas
6. Se selecciona el mejor modelo
7. Se evalúa
8. Es un proceso iterativo, en cualquier momento se puede volver atrás



Aprendizaje supervisado

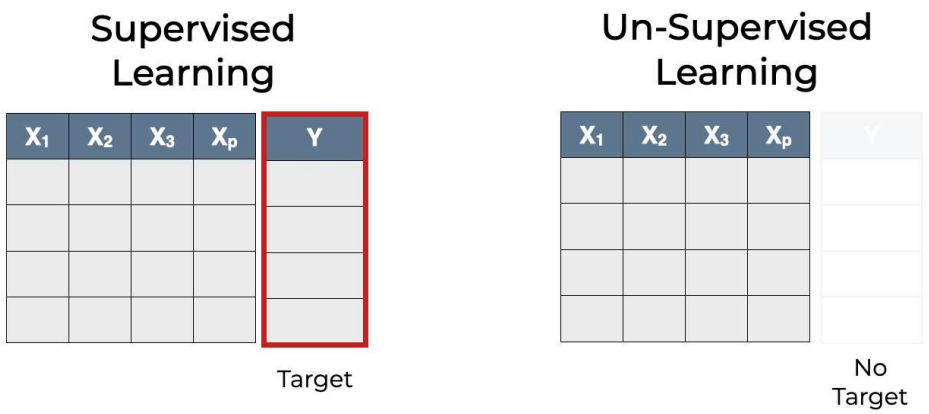
El aprendizaje supervisado es una familia de algoritmos que utiliza un conjunto de datos etiquetados para su entrenamiento y cuyo objetivo es hacer predicciones sobre nuevos datos.

Donde el conjunto de datos etiquetados es de la forma:

$$D = \{(x_i, y_i)\}_{i=1}^{N}$$

Siendo x_i el vector de características para la observación i, e y_i el valor del atributo objetivo para dicha observación.

Se puede comprobar la diferencia entre aprendizaje supervisado y no supervisado en base a los datos de entrada:



La calidad del modelo depende de los datos, por lo que el paso previo de preparación de datos es fundamental. Una clasificación sencilla de los datos podría ser:

- Datos numéricos, que tienen significado como medida. A su vez, pueden ser:

    - Discretos, enteros como el número de habitaciones de una casa.
    - Continuos, como la temperatura.

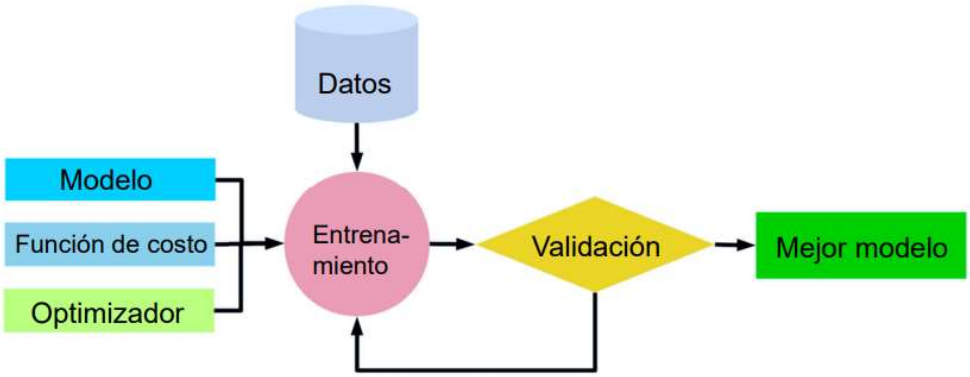
- Datos categóricos, que representan categorías. Pueden ser números, pero no tienen valor matemático. Por ejemplo, el estado civil de una persona.

- Datos ordinales, donde las distintas categorías pueden ser numeradas, como, por ejemplo, la clase de un billete. Muchas veces se tratan como categóricos.

El conjunto de datos se divide en datos de entrenamiento y datos de prueba. En aprendizaje supervisado sí que tenemos, para el conjunto de entrenamiento y de prueba, el valor de la variable objetivo que se quiere predecir.

Normalmente los datos se reparten en un 70% datos de entrenamiento y un 30% datos de prueba, pero se puede variar la proporción según el caso.

Los datos de entrenamiento son los datos que se usan para entrenar el modelo, mientras que los datos de prueba sirven para evaluar dicho modelo. La calidad del modelo depende de los datos, por lo que el paso previo de preparación de datos es fundamental.

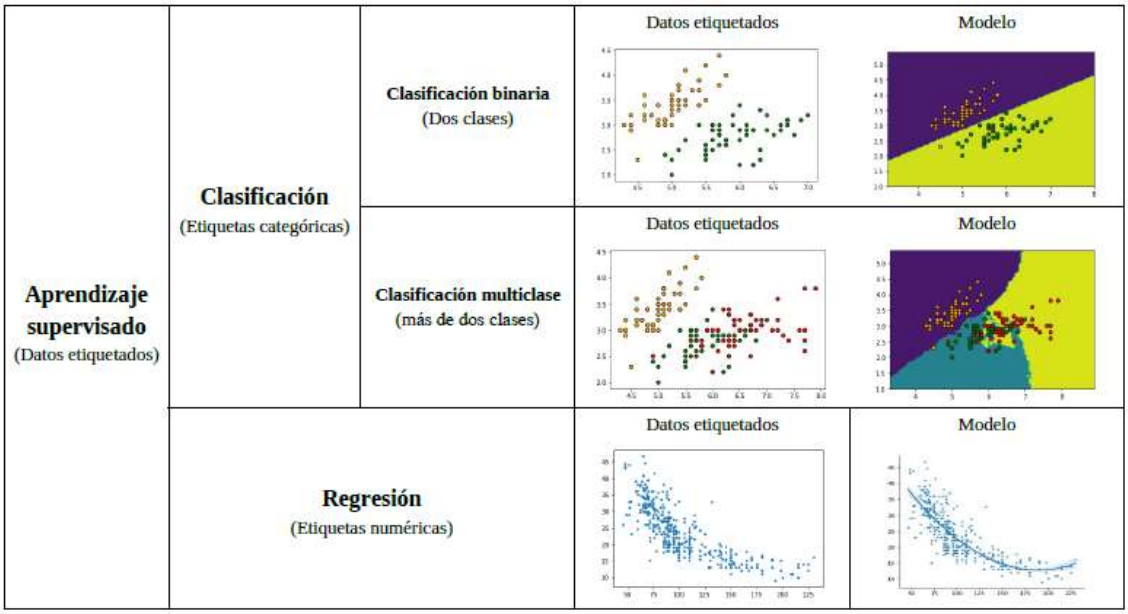


Algunos de los problemas con los que nos podemos encontrar:

- Sobreajuste u overfitting: Son modelos complejos que se ajusta tan milimétricamente al conjunto de datos a partir del cual se han creado, que pierden gran parte de su poder predictivo, y ya no son útiles para otros conjuntos de datos.

- Subajuste o underfitting: Ocurre cuando el conjunto de datos de entrenamiento es insuficiente, con ruido en alguna de sus dimensiones o, en definitiva, poco representativo. Como consecuencia, nos lleva a un modelo excesivamente simple, con poco valor predictor.

Existen dos tipos de aprendizaje supervisado: clasificación y regresión.

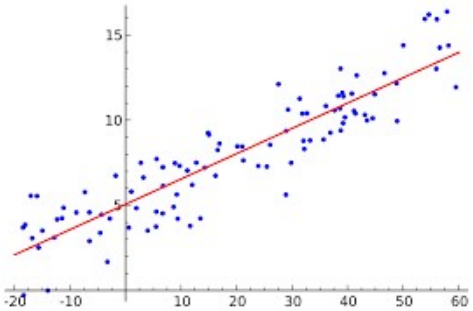


Regresión lineal

Modelo de regresión, como su propio nombre indica. Partimos de la suposición más sencilla, regresión lineal simple, aquella en que los datos son unidimensionales:

$$D = \{(x_j, y_j) : j = 1, \ldots, N\}, \text{ donde } x_j, y_j \in \mathbb{R}$$

Y en la que existe una relación lineal entre X e Y.



Por tanto, se trata de encontrar la recta que mejor se adapte a los datos:

$$Y = \beta_0 + \beta_1 X$$

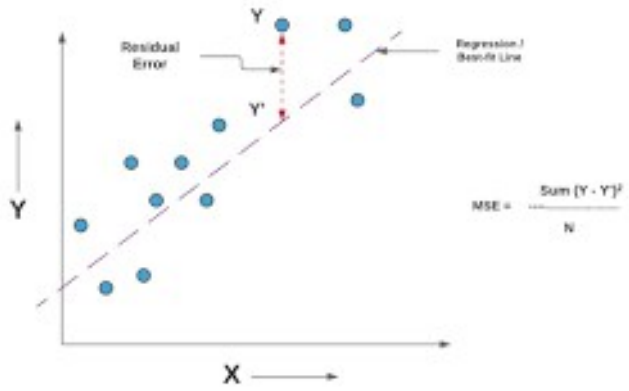
β_1 representa la pendiente de la recta y β_0 el punto de corte con el eje y. Estos dos coeficientes son los valores que se obtienen una vez que se ha entrenado el modelo.

Para evaluar cómo de buenos son estos coeficientes se usa una función de coste que mide la diferencia entre las predicciones de un modelo y los valores reales, cuantificando el error y la calidad del modelo.

Esta función puede ser el error cuadrático medio, que calcula la media de las diferencias al cuadrado entre el valor predicho y el real:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Donde y_i es el valor real y $\hat{y}_i$ el predicho. La función de coste o función de pérdida mide la diferencia entre el valor real y el pronosticado.



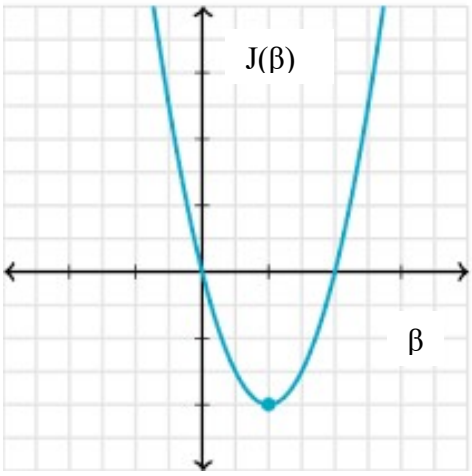
Se trata de encontrar los coeficientes que minimicen dicha función de coste. Para ello existen muchos optimizadores, pero nos podemos centrar en el descenso del gradiente.

El descenso del gradiente es un algoritmo de optimización que se usa para entrenar modelos de machine learning y redes neuronales. El descenso del gradiente es un algoritmo de optimización empleado para encontrar el mínimo de una función.

Simplificando para entender el proceso, vamos a suponer que β_0 es cero, con lo que la función de coste quedaría:

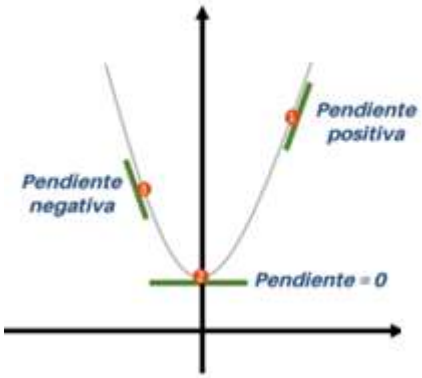
$$J(\beta) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 = \frac{1}{N}\sum_{i=1}^{N}(\beta x_i - y_i)^2$$

Como una función que depende de β, si representamos dicha función:



El valor de β donde la función tiene su valor mínimo es el que estamos buscando.

El gradiente representa la pendiente de la recta tangente a la gráfica de una función en un punto dado y matemáticamente recibe el nombre de derivada.



El algoritmo de descenso del gradiente es iterativo (es decir que se repite el mismo procedimiento varias veces) y requiere de dos parámetros:

- El número de iteraciones, es decir cuántas veces se repetirá el procedimiento.
- La tasa de aprendizaje (o *learning rate*), conocido como el parámetro α.

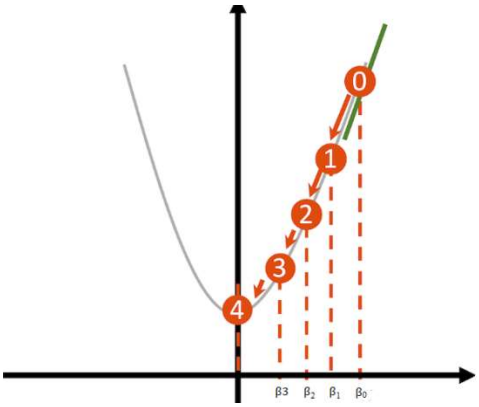
Una vez definidos estos dos parámetros, el algoritmo procede de la siguiente manera:

1. Escoge de forma aleatoria el valor inicial de la variable β.
2. En cada iteración actualiza el valor de β usando la siguiente ecuación:

$$\beta = \beta - \alpha \nabla f(\beta)$$

   Donde $\nabla f(\beta)$ es el gradiente de la función evaluada en el punto β y α es la tasa de aprendizaje, hiperparámetro que controla el tamaño de los pasos que da el algoritmo en cada iteración.

   Este paso se repite hasta completar el número de iteraciones definido por el usuario.



La librería scikit-learn (sklearn) proporciona una serie de herramientas para el aprendizaje automático y el modelado estadístico que incluyen clasificación, regresión, agrupación y reducción de dimensionalidad a través de una interfaz de consistencia en Python.

Esta biblioteca, que está escrita en gran parte en Python, se basa en NumPy, SciPy y Matplotlib.

En el caso de tener más de una variable ya hablamos de regresión lineal múltiple. LinearRegression entrena un modelo lineal con coeficientes que minimizan la suma de los cuadrados entre los valores observados y los predichos:

Atributos	**coef_:** ***array of shape (n_features)*** Son los coeficientes para un problema de regresión múltiple. **intercept_:** Término independiente del modelo lineal. **n_features_in_:** Número de características.
Métodos	**fit**: Entrena el modelo. **predict**: Realiza la predicción. **score**: Devuelve el coeficiente de determinación de los datos de prueba. **get_params**: Obtiene los parámetros

Un ejemplo:

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
# dot multiplica matrices
y = np.dot(X, np.array([1, 2])) + 3                    # y = 1 * x_0 + 2 * x_1 + 3
reg = LinearRegression().fit(X, y)
reg.score(X, y)
  1.0
reg.coef_
  array([1., 2.])
reg.intercept_
  3.0...
reg.predict(np.array([[3, 5]]))
  array([16.])

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
```

En términos generales, un modelo lineal intenta encontrar la relación entre las variables de entrada y la variable de salida. Sin embargo, si un modelo lineal tiene demasiados parámetros o si los datos son muy ruidosos, puede suceder que el modelo se ajuste demasiado a los datos de entrenamiento, produciéndose un sobreajuste.

Para evitar este problema, los modelos lineales regularizados añaden un término extra para penalizar los valores de los coeficientes que son demasiado grandes. Estos modelos son regresiones lineales, pero con la adición de un término de regularización. Los dos tipos de modelos son:

- Modelo lineal regularizado de Lasso (*L1*): Añade una penalización igual al valor absoluto de la magnitud de los coeficientes. Puede resultar en coeficientes iguales a cero, lo que indica que la característica correspondiente no se utiliza en el modelo.

- Modelo lineal regularizado de Ridge (*L2*): Añade una penalización igual al cuadrado de la magnitud de los coeficientes. Esto tiende a reducir los coeficientes, pero no los hace exactamente cero, por lo que todas las características se mantienen en el modelo.

Regresión logística

Pese a su nombre es un modelo para resolver problemas de clasificación binaria. Es un método estadístico que trata de modelar la probabilidad de una variable cualitativa binaria (dos posibles valores) en función de una o más variables independientes.

Como ya vimos en regresión lineal:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} + \epsilon_i$$

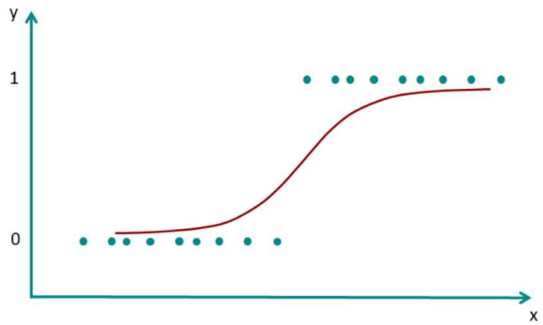
Donde β son los coeficientes que ponderan el peso de cada variable y ε el error. Ahora bien, la variable objetivo es una variable binaria (0 y 1), pero y_i no tiene por qué estar dentro del intervalo [0,1]. Para evitar este problema, la regresión logística transforma el valor devuelto por la regresión lineal con una función cuyo resultado siempre está comprendido entre 0 y 1. Existen varias funciones que cumplen esta descripción, una de las más utilizadas es la función logística (también conocida como función sigmoide):

$$\text{sigmoide} = \sigma(y) = \frac{1}{1 + e^{-y}}$$

Para valores de y positivos muy grandes, e^{-y} es aproximadamente 0, por lo que el valor de la función sigmoide es 1. Para valores de y negativos muy grandes, e^{-y} tiende a infinito, por lo que el valor de la función sigmoide es 0.

$$\hat{y} = \boxed{b_1 \cdot x_1 + b_2 \cdot x_2 + \ldots + b_k \cdot x_k + a}$$

$$f(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\boxed{(b_1 \cdot x_1 + \ldots + b_k \cdot x_k + a)}}}$$



La clase LogisticRegression:

Atributos	**coef_ : nd*array of shape (1, n_features)*** Son los coeficientes de las características. **intercept_ :** Término independiente del modelo. **n_features_in_ :** Número de características.
Métodos	**fit**: Entrena el modelo. **predict**: Realiza la predicción. **predict_proba**: Probabilidades estimadas. **score**: Devuelve el accuracy. **get_params**: Obtiene los parámetros

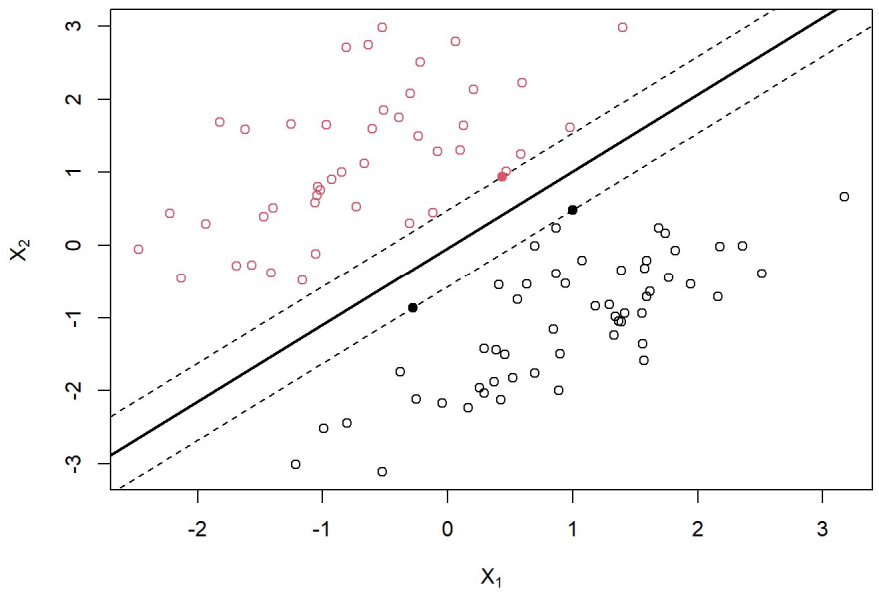
Un ejemplo:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
clf = LogisticRegression(random_state=0).fit(X, y)
clf.predict(X[:2, :])
clf.predict_proba(X[:2, :])
   array([[9.8...e-01, 1.8...e-02, 1.4...e-08], [9.7...e-01, 2.8...e-02, ...e-08]])
clf.score(X, y)
  0.97...
```

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

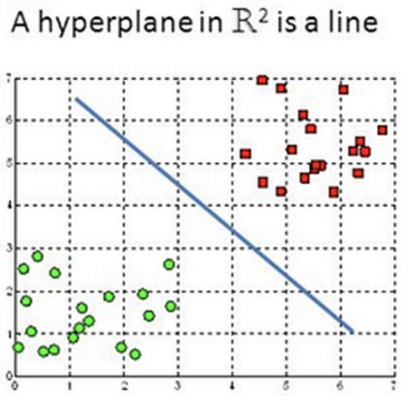
Support vector machines (SVM)

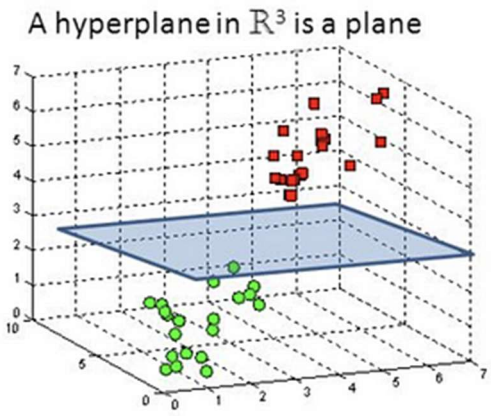
En un principio las máquinas de vector soporte se plantearon como un modelo para resolver problemas de clasificación binaria. El objetivo es encontrar el hiperplano en un espacio de N dimensiones que mejor separe los datos.



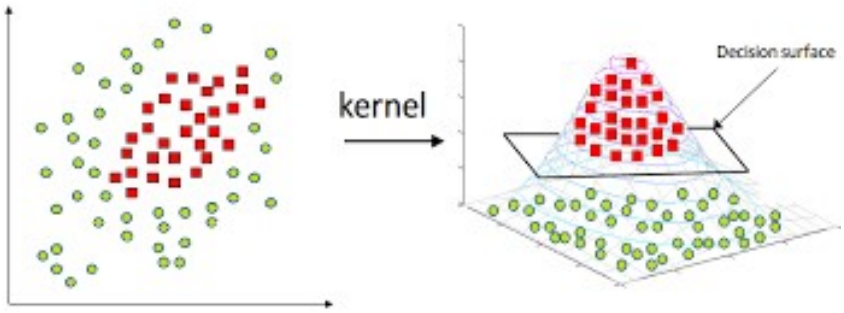
Dado un hiperplano de separación, se calculan las distancias a todos los datos de entrenamiento y se define el margen como la menor de esas distancias. Se trata de seleccionar, de entre los infinitos hiperplanos, aquel que tiene el margen mayor. En la imagen la línea sólida es el hiperplano y las líneas discontinuas son las vectores de soporte.

En un espacio p-dimensional, un hiperplano se define como un subespacio de dimensiones p – 1. En un espacio de dos dimensiones el hiperplano tiene una dimensión, esto es, es una línea, y si el espacio es de tres dimensiones el hiperplano es un plano.





SVM pertenece a una clase de algoritmos de denominados métodos kernel, donde se puede utilizar una función de kernel para transformar las características. Las funciones de kernel asignan los datos a un espacio dimensional diferente, que suele ser superior, con la expectativa de que resulte más fácil separar las clases después de esta transformación.



Simplificando, algunos tipos de kernel son:

- Linear Kernel: Calcula el producto escalar de los dos vectores y añade una constante opcional. No transforma el dataset a un espacio de mayor dimensionalidad.

  $$K(x_1, x_2) = x_1^T x_2 + c$$

  Es aplicable cuando la relación entre características y variable objetivo es lineal.

- Polynomial Kernel: Para espacios de entrada no lineales. Esto lo hace usando funciones polinómicas de las características originales.

  $$K(x_1, x_2) = (x_1^T x_2 + c)^d$$

  Donde x_1 y x_2 son los vectores del espacio original (x_1^T es el vector transpuesto, esto es, si es un vector fila, la transpuesta es un vector columna). En este caso tenemos dos parámetros: c y d, siendo d el grado del polinomio.

  Un valor grande de c puede dar lugar a valores de error bajos pero a cambio existe el riesgo de overfitting, por el contrario, un valor pequeño de c dará un error más elevado, pero disminuye el riesgo de overfitting. El valor de d controla la complejidad del modelo, a mayor valor mayor complejidad y más riesgo de overfitting, mientras que un valor pequeño puede no alcanzar los resultados esperados. Se trata de encontrar el mejor valor para los parámetros.

  En https://www.pycodemates.com/2022/10/svm-kernels-polynomial-kernel.html hay un ejemplo un interesante.

- Radial Basis Function (RBF), también conocido como Gaussian Kernel se utiliza en espacios de entrada complejos y no lineales. Es uno de los kernels más usados.

  $$K(X_1, X_2) = exp(-\frac{||X_1 - X_2||^2}{2\sigma^2})$$

  Donde σ es la varianza y $||X_1 - X_2||$ la distancia entre dos puntos. Un valor alto de σ se adapta perfectamente al dataset pero ocasiona overfitting, un valor de 0,1 es suele considerar adecuado.

SVC (clasificación)

La clase SVC (Support Vector Classifier) de Scikit-Learn implementa un clasificador basado en máquinas de soporte vectorial.

Este algoritmo es aplicable a problemas de clasificación multiclase.

Parámetros	**C:** ***default=1.0*** Parámetro de regularización. **kernel:** ***{"linear", "poly", "rbf", "sigmoid", "precomputed"}, default="rbf"*** Especifica el tipo de kernel a ser usado en el algoritmo. **Degree:** ***default=3*** Grado de la función de núcleo polinómico ("poly"). Ignorado por todos los otros núcleos. **Gamma:** ***{"scale", "auto"} o flotante, default="scale"*** Coeficiente de núcleo para "rbf", "poly" y "sigmoid". • si gamma='scale' (valor predeterminado), entonces se usa 1 / (n_features * X.var()) como valor de gamma, • si es "auto", utiliza 1 / n_features. **coef0:** ***default=0.0*** Término independiente en la función de kernel. Sólo es significativo en "poly" y "sigmoid".
Atributos	**coef_:** ***ndarray*** Ponderaciones asignadas a las características. Solo para kernel lineal. **support_vectors_:** ***ndarray de forma (n_SV, n_features)*** Vectores de apoyo. **n_support:** ***ndarray de forma (n_classes,)*** Número de vectores de soporte para cada clase.
Métodos	**fit**: Ajusta el modelo de acuerdo a los datos de entrenamiento **predict**: Realiza la clasificación **score**: Devuelve la precisión media en los datos de prueba **get_params**: Obtiene los parámetros

https://qu4nt.github.io/sklearn-doc-es/modules/generated/sklearn.svm.SVC.html

Para que el modelo funcione correctamente hay que seleccionar los hiperparámetros adecuadamente.

Con respecto al parámetro de regularización, a las muestras que son clasificadas erróneamente se les asocia una penalización que depende de lo lejos que se encuentren del área correcta en la que deberían estar incluidas, siendo el objetivo de estos modelos el obtener el mayor margen posible con la mínima penalización total.

En estos clasificadores, se introduce un parámetro de regularización C que controla el equilibrio entre el doble objetivo de maximizar el margen y minimizar el error cometido. C multiplica a los errores cometidos, de forma que un valor bajo de C tiende a penalizar poco los errores, permitiendo márgenes mayores a costa de cometer más errores

(baja regularización), mientras que un valor alto de C penaliza mucho los errores, llevando al modelo a cometer menos errores a costa de un margen menor (alta regularización).

Ejemplo:

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])
y = np.array([1, 1, 2, 2])
clf = make_pipeline(StandardScaler(), SVC(gamma='auto’))
clf.fit(X, y)
  Pipeline(steps=[('standardscaler', StandardScaler()), ('svc', SVC(gamma='auto'))])
print(clf.predict([[-0.8, -1]]))
  [1]
```

https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

SVR. Regresión

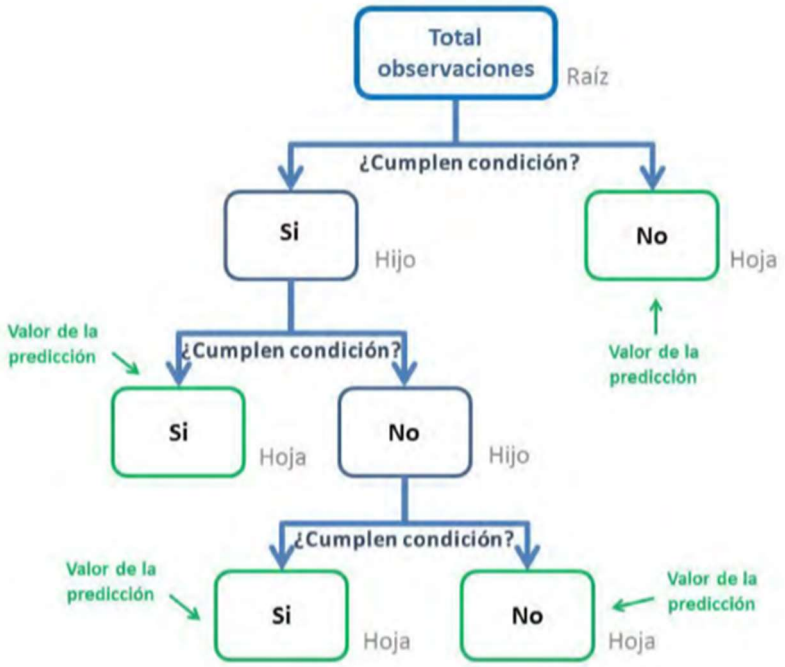
Ejemplo:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVR

X = np.sort(5 * np.random.rand(40, 1),  axis=0)
y = np.sin(X).ravel()
y[::5] += 3 * (0.5 - np.random.rand(8))

svr = SVR(kernel='rbf')
svr.fit(X, y)
y_pred = svr.predict(X)

plt.scatter(X, y, color='darkorange’,  label='data')
plt.plot(X, y_pred, color='cornflowerblue’,  label='prediction')
plt.legend()
plt.show()
```

https://www.geeksforgeeks.org/support-vector-regression-svr-using-linear-and-non-linear-kernels-in-scikit-learn

Árboles de decisión

Es un modelo, que una vez entrenado, proporciona un grafo con estructura de árbol para hacer las predicciones.

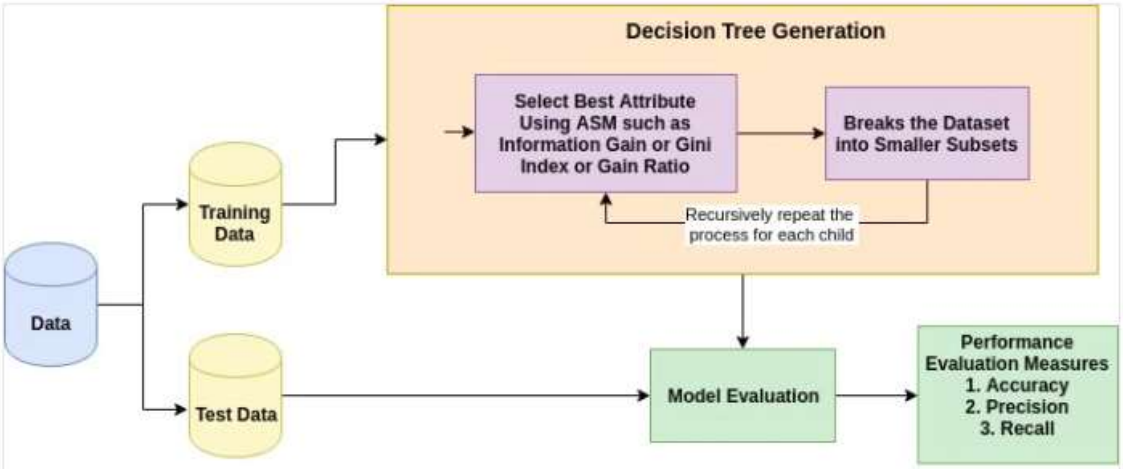


Los árboles de decisión son una técnica muy popular debido a una serie de ventajas:

- No requieren mucho tiempo de preprocesado y limpieza de datos, ya que pueden funcionar con diferentes tipos de datos
- Pueden manejar conjuntos de datos incompletos, es decir, conjuntos de datos que tienen valores nulos.
- Son fáciles de entender y visualizar

Por el contrario, son muy sensibles a pequeños cambios en el conjunto de entrenamiento o a la presencia de ruido. Si el conjunto de datos es grande se necesita más tiempo para entrenar el algoritmo y, además, tienden a adaptarse a los datos, lo que se traduce en overfitting.

Básicamente, funciona de la siguiente forma:



Esto es, se selecciona un atributo y se hace del mismo un nodo del árbol, lo cual redunda en una división de dataset. Esto se repite hasta que todas las instancias estén clasificadas o hasta que no haya más atributos o hasta que no haya más instancias.

La forma de selección de los atributos está basada en:

- Ganancia de información: concepto relacionado con la entropía. La entropía se utiliza para medir la impureza de un conjunto de datos. Cuanto mayor sea la entropía, mayor será la incertidumbre en los datos y, por lo tanto, mayor será la impureza. En el contexto de un árbol de decisión, se busca dividir el conjunto de datos en subconjuntos más puros, es decir, con menor entropía.

  Los valores de entropía pueden estar comprendidos entre 0 y 1. Si todas las muestras del conjunto de datos pertenecen a una clase, entonces la entropía será igual a cero.

  La ganancia de información representa la diferencia en la entropía antes y después de una división en un atributo dado. El atributo con la mayor ganancia de información producirá la mejor división, ya que está haciendo el mejor trabajo al clasificar los datos de entrenamiento de acuerdo con su clasificación de destino. La ganancia de información suele representarse con la siguiente fórmula:

$$\text{Gain}(S, A) = \text{Entropía}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropía}(S_v)$$

  Donde:

  - A es el atributo por el cual se está dividiendo el conjunto de datos S
  - Values(A) son los posibles valores que puede tomar el atributo A
  - Sv es el subconjunto de S en el cual el atributo A toma el valor v
  - |S| y |Sv| son el número de ejemplos en S y Sv

  Se toma la entropía del conjunto de datos original y se le resta la suma ponderada de las entropías de los subconjuntos resultantes después de dividir el conjunto por un atributo específico. El atributo que proporciona la mayor ganancia de información se elige como el atributo de división en un árbol de decisión.

- Índice de Gini:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2$$

  Donde:

  - S es el conjunto de datos
  - c es el número de clases en el conjunto de datos
  - p_i es la proporción de instancias de la clase i en el conjunto de datos

  El objetivo en un árbol de decisión es reducir la impureza de Gini al dividir el conjunto de datos en subconjuntos más puros.

  La impureza de Gini proporciona una medida de cuán impuro o mezclado está un conjunto de datos en términos de sus clases, y se utiliza en el proceso de construcción de árboles de decisión para elegir la mejor manera de dividir los datos.